BaseAjax

BaseAjax یک HTTP client برای JavaScript سبک و بدون وابستگی زمان اجرا برای CommonJS و ESM است. این کتابخانه در مرورگر، HTTP و برای دریافت و ارسال درخواست‌های middleware مدیریت خطا و افزودن، های اختیاری طراحی شده است.

فهرست

- نصب و شروع سریع
- ورودی‌ها و خروجی‌های بسته
- ارسال درخواست
- پیکربندی
- احراز هویت و CSRF
- تلاش مجدد و لغو درخواست
- پاسخ‌ها و خطاها

- Interceptorها و ثبت رخداد
- Transport سفارشی

- پلاگین‌ها
- ساخت و بررسی پروژه

نصب و شروع سریع

```
npm install base-ajax
```

این مثال یک client می‌سازد، آدرس پایه را تنظیم می‌کند و JSON دریافت می‌کند:

```
import { createClient } from "base-ajax";

const api = createClient({
  baseURL: "https://api.example.com",
  responseType: "json",
});

const users = await api.get("/users", { page: 1 });
```

برای درخواست‌های GET و HEAD، آرگومان دوم query parameters است. برای PATCH، PUT، POST و DELETE آرگومان دوم بدنه درخواست است.

ورودی‌ها و خروجی‌های بسته

```
// ESM
import BaseAjax, { ApiError, createClient } from "base-ajax";

// CommonJS
const BaseAjax = require("base-ajax");
```

برای `<script>` در مرورگر از `dist/baseAjax.umd.min.js` استفاده کنید. کتابخانه را در `window.BaseAjax` خواهید دید. اگر به پلاگین‌های مرورگر نیاز دارید، `dist/plugins.umd.js` را نیز بارگذاری کنید؛ آن‌ها در `window.BaseAjaxPlugins` قرار می‌گیرند.

ورودی پلاگین‌ها برای ESM و CommonJS:

```
import { cache, compose, dedupe } from "base-ajax/plugins";
```

ارسال درخواست

متدهای کوتاه

```
const profile = await api.get("/users/42");
const page = await api.get("/users", { page: 2, active: true });
const created = await api.post("/users", { name: "Ada" });
const updated = await api.patch("/users/42", { name: "Grace" });
await api.delete("/users/42");
```

متدهای موجود `delete` ، `patch` ، `put` ، `post` ، `head` ، `get` و `options` هستند. متدهای `get` و `head` داده آرگومان دوم را به query string تبدیل می‌کنند. آرایه‌ها به چند پارامتر هم‌نام تبدیل می‌شوند؛ مقدارهای `null` و `undefined` نادیده گرفته می‌شوند.

برای سازگاری با الگوی قدیمی کتابخانه، `callAPI(url, method, data, options, token)` نیز موجود است. این متد به‌طور پیش‌فرض JSON می‌خواهد و token آرگومان آخر را فقط وقتی `options.withAuth` برابر `true` باشد استفاده می‌کند.

متد عمومی

برای کنترل دقیق‌تر از `request` استفاده کنید:

```
const data = await api.request({
  url: "/reports",
  method: "POST",
  data: { from: "2026-01-01", to: "2026-01-31" },
  headers: { "X-Report-Mode": "monthly" },
});
```

`request` یک رشته URL یا یک شیء تنظیمات می‌پذیرد. برای GET/HEAD از `params` و برای متدهای دیگر از `data` استفاده کنید.

فرمت بدنه

- شیءها و آرایه‌های JavaScript به JSON تبدیل می‌شوند و `Content-Type` مناسب می‌گیرند.
- رشته بدون تبدیل ارسال می‌شوند و typed array ، `ArrayBuffer` ، `Blob` ، `URLSearchParams` ، `FormData`.
- برای `FormData` ، مرورگر مرز multipart را در `Content-Type` قرار می‌دهد.
- مقدارهای query از `URLSearchParams` ، رشته query، شیء، آرایه، تاریخ و مقدارهای ساده پشتیبانی می‌کنند.

Progress و متدهای جایگزین

`onUploadProgress` و `onDownloadProgress` callbackهایی دریافت می‌کنند که شیء `{ loaded, total, lengthComputable }` را انتخاب می‌کند. Fetch این XHR وجود این ، `auto` استفاده می‌شود؛ در حالت XHR نوع transport از progress می‌گیرند. برای callbackها را گزارش نمی‌کند.

برای سرورهایی که PUT، PATCH یا DELETE را مستقیم نمی‌پذیرند، `methodOverride` پیش‌فرض `"header"` است و درخواست‌های trusted این متدها را با POST و `X-HTTP-Method-Override` header می‌فرستد. مقدار `false` این رفتار را خاموش می‌کند. مقدار `"body"` نام متد اصلی را به فیلد `_method` در بدنه قابل‌ویرایش اضافه می‌کند؛ برای داده‌های غیرقابل‌ویرایش، به روش header برمی‌گردد. نام header، فیلد و متدهای مشمول با `methodOverrideField` ، `methodOverrideHeader` و `methodOverrideMethods` قابل تغییرند.

پیکربندی

تنظیمات را می‌توان هنگام ساخت client یا برای هر درخواست مشخص کرد. تنظیمات درخواست بر تنظیمات client اولویت دارند. `configure(options)` تنظیمات client جاری را تغییر می‌دهد و `create(options)` یک client جدا با تنظیمات ترکیبی می‌سازد.

```
const api = createClient({
  baseURL: "https://api.example.com",
  timeout: 8000,
  headers: { Accept: "application/json" },
});

api.configure({ timeout: 5000 });
const isolatedApi = api.create({ baseURL: "https://other.example.com" });
```

نام headerها بدون حساسیت به بزرگی حروف ادغام می‌شوند. مقدار `null` در `headers` یک header پیشین را حذف می‌کند.

تنظیمات اصلی

تنظیم	پیش‌فرض	شرح
`baseURL`	`""`	پیشوند URL برای مسیرهای نسبی
`timeout`	`12000`	حداکثر زمان هر تلاش، بر حسب میلی‌ثانیه؛ `0`، `null` و `Infinity` غیرفعال می‌کنند
`headers`	`{}`	هدرهای پیش‌فرض
`responseType`	`"auto"`	یکی از `blob`، `text`، `json`، `arraybuffer` و `auto`
`transport`	`"auto"`	یکی از `jquery`، `xhr`، `fetch` یا `auto` یا تابع سفارشی
`allowedOrigins`	`[]`	origin‌های مجاز برای ارسال اعتبارنامه
`origin`	`null`	تشخیص داده می‌شود `location.origin` صفحه؛ در صورت خالی بودن از origin
`fullResponse`	`false`	در صورت فعال بودن، به‌جای `data` کل پاسخ برگردانده می‌شود
`bustCache`	`false`	برای GET/HEAD پارامتر زمان‌دار `_` به URL اضافه می‌کند
`logLevel`	`"warn"`	سطح ثبت رخداد: `error`، `warn`، `info`، `debug` یا `silent`
`logger`	`null`	مقصد سفارشی دارای متدهای `error`، `warn`، `info`، `debug` یا `log`
`xRequestedWith`	`true`	برای درخواست‌های trusted، header استاندارد `X-Requested-With` را اضافه می‌کند
`correlationHeader`	`null`	نام header اختیاری برای ارسال `correlationId`
`idempotencyKey`	تنظیم نمی‌شود	رشته دلخواه یا `true` برای تولید شناسه خودکار درخواست

در حالت `transport: "auto"`، اگر progress درخواست شده باشد XHR انتخاب می‌شود؛ در غیر این صورت Fetch و سپس XHR بر اساس موجودبودن آن‌ها انتخاب می‌شوند. jQuery باید صریحاً با `transport: "jquery"` انتخاب شود. `methodOverride` به‌صورت پیش‌فرض `"header"` است، فقط برای originهای trusted اعمال می‌شود و متدهای پیش‌فرض آن PATCH، PUT و DELETE هستند.

گزینه‌های سازگاری قدیمی `retryDelay`، `retryCount`، `dataType` و `cache` هنوز پذیرفته می‌شوند؛ در کد جدید از `responseType`، `retry.delay`، `retry.count` و `bustCache` استفاده کنید.

برای کنترل رفتار یک درخواست بدون تغییر client، گزینه‌های زیر را به همان درخواست بدهید:

گزینه	کاربرد
`signal`	لغو با `AbortController`
`abortKey`	لغو درخواست قبلی با کلید یکسان
`headers`	افزودن header؛ `null` یک header پیش‌فرض را حذف می‌کند
`contentType`	تعیین صریح `Content-Type` بدنه
`token` و `withAuth`	فعال/غیرفعال‌کردن احراز هویت یا تعیین token همین درخواست
`credentials`	تعیین سیاست credentialهای Fetch/XHR
`skipRefresh`	جلوگیری از تلاش refresh برای این درخواست

گزینه	کاربرد
`validateStatus(status)`	تعیین statusهایی که پاسخ موفق محسوب می‌شوند
`onUploadProgress` و `onDownloadProgress`	دریافت رویداد progress از XHR
`correlationId`	تعیین شناسه این درخواست؛ با `correlationHeader` قابل ارسال به سرور است

انواع پاسخ

`responseType: "auto"` بر اساس `Content-Type` پاسخ JSON را parse می‌کند؛ در غیر این صورت متن برمی‌گرداند. `json` ، `text` ، `blob` ، `arraybuffer` و `null` دارند. نوع پاسخ را صریح می‌کنند. پاسخ‌های 204 و 205 بدون بدنه،

احراز هویت و CSRF

تنظیمات احراز هویت در `auth` قرار می‌گیرند:

تنظیم	پیش‌فرض	شرح
`auth.mode`	`"cookie"`	روش `memory` ، `cookie` ، `none` یا token: `storage`
`auth.header`	`"Authorization"`	نام header توکن
`auth.scheme`	`"Bearer"`	پیشوند token؛ رشته خالی یعنی ارسال token بدون پیشوند
`auth.storage`	`"local"`	در حالت storage، یکی از `local` یا `session`
`auth.storageKey`	`"token"`	کلید ذخیره token
`auth.storageArea`	`null`	شیء سفارشی سازگار با Storage
`auth.getToken`	`null`	تابع دریافت token؛ می‌تواند Promise برگرداند
`auth.setToken`	`null`	تابع سفارشی ذخیره یا پاک‌کردن token
`auth.refresh`	`null`	تابع async برای دریافت token تازه
`auth.refreshUrl`	`null`	URL جایگزین برای درخواست refresh
`auth.refreshMethod`	`"POST"`	متد درخواست refresh
`auth.tokenField`	`"accessToken"`	نام فیلد token در پاسخ refresh
`auth.onUnauthorized`	`null`	callback پس از خطای 401 و پایان refresh

در حالت `memory` ، از `getToken` ، `setToken` و `clearToken` استفاده کنید:

```
const api = createClient({
  baseURL: "https://api.example.com",
  auth: { mode: "memory" },
});

api.setToken("ACCESS_TOKEN");
const profile = await api.get("/profile");
api.clearToken();
```

برای غیرفعال‌کردن token در یک درخواست، `withAuth: false` بدهید. برای الزام token در یک درخواست، `withAuth: true` بگذارید. همچنین می‌توانید `token` همان درخواست را مشخص کنید.

وقتی پاسخ 401 دریافت شود، در صورت تنظیم `auth.refresh` یا `auth.refreshUrl` ، حداکثر یک بار برای همان درخواست اجرا می‌شود و درخواست اصلی دوباره فرستاده می‌شود. refreshهای همزمان بین درخواست‌ها مشترک می‌شوند. درخواست refresh داخلی، احراز هویت و retry ندارد. پس از شکست refresh، خطای 401 اصلی رد می‌شود و `onUnauthorized` اجرا می‌شود.

در حالت token ، `storage` با JavaScript قابل خواندن است و در برابر XSS محافظت نمی‌شود. برای token حساس، storage را تنها با آگاهی از این ریسک انتخاب کنید.

برای CSRF از تنظیمات `csrf` استفاده کنید. نام پیش‌فرض cookie برابر `XSRF-TOKEN` ، header برابر `X-XSRF-TOKEN` و meta برابر `csrf-token` است. در درخواست‌های غیرایمن، token ابتدا از cookie و سپس از meta خوانده می‌شود. این قابلیت فقط در محیط دارای `document` قابل استفاده است.

Origin و اعتبارنامه

BaseAjax فقط URLهای HTTP و HTTPS را می‌پذیرد. origin صفحه و `baseURL` trusted origin هستند؛ برای مقصدهای دیگر آن‌ها را در `allowedOrigins` مقصد وارد کنید. اگر ارسال صریح trusted نباشد، `token` یا `withAuth: true` با خطای `UNTRUSTED_ORIGIN` رد می‌شود. `credentials` را در تنظیمات درخواست تعیین کنید؛ می‌توانید `omit` و برای مقصد دیگر `include` برابر trusted پیش‌فرض برای مقصد credential است.

تلاش مجدد و لغو درخواست

`retry` آن را خاموش می‌کند `false` باشد. مقدار retry می‌تواند شیء سیاست یا مقدار عددی تعداد.

گزینه	پیش‌فرض	شرح
`count`	`0`	تعداد تلاش‌های مجدد بعد از درخواست اول
`delay`	`1000`	تأخیر اولیه بر حسب میلی‌ثانیه
`maxDelay`	`30000`	بیشینه تأخیر backoff
`factor`	`2`	ضریب افزایش تأخیر
`jitter`	`true`	تصادفی‌سازی تأخیر برای پخش درخواست‌ها
`methods`	`GET` ، `HEAD` ، `OPTIONS`	متدهای قابل retry بدون idempotency key
`statuses`	`408` ، `425` ، `429` ، `500` ، `502` ، `503` ، `504`	statusهای HTTP قابل retry
`onTimeout`	`true`	timeout در retry
`onNetworkError`	`true`	در خطای شبکه retry
`respectRetryAfter`	`true`	رعایت header پاسخ `Retry-After`
`maxRetryAfter`	`60000`	بیشینه زمان پذیرفته‌شده از `Retry-After`

درخواست فقط زمانی retry می‌شود که نوع خطا و متد با سیاست سازگار باشند. برای retry متدهای دیگر، `idempotencyKey` تنظیم کنید. `timeout` برای هر تلاش جداگانه محاسبه می‌شود.

برای لغو، `signal` از `AbortController` بدهید. `abortKey` نیز درخواست قبلی دارای همان کلید را لغو می‌کند:

```
const controller = new AbortController();
const pending = api.get("/large-report", undefined, {
  signal: controller.signal,
});
controller.abort();

// درخواست قبلی را لغو می‌کند abortKey درخواست بعدی با همان.
api.get("/search", { q: "first" }, { abortKey: "search" });
api.get("/search", { q: "second" }, { abortKey: "search" });
```

پاسخ‌ها و خطاها

به‌طور پیش‌فرض، متدها فقط `response.data` را برمی‌گردانند. برای دریافت status و headerها، `fullResponse: true` تنظیم کنید:

```
const response = await api.get("/users", undefined, { fullResponse: true });
console.log(
  response.data,
  response.status,
  response.headers,
  response.attempts,
);
```

پاسخ کامل شامل `correlationId` ، `method` ، `url` ، `headers` ، `statusText` ، `status` ، `data` و `attempts` است.

خطاهای کتابخانه از نوع `ApiError` هستند. فیلدهای اصلی آن ، `method` ، `url` ، `headers` ، `data` ، `statusText` ، `status` ، `code` ، `attempts` ، `correlationId` و `cause` هستند. `response` نام مستعار قدیمی `data` است.

برای تشخیص خطا از `isApiError(error)` یا `error instanceof ApiError` استفاده کنید. ویژگی‌های کمکی ، `isTimeout` ، `isHttp` ، `isNetwork` و `isAborted` نوع خطا را مشخص می‌کنند. `()toJSON` خلاصه‌ای از خطا برمی‌گرداند و بدنه پاسخ را در خروجی قرار نمی‌دهد.

کدهای خطا:

کد	معنا
`HTTP_ERROR`	پذیرفته نشده است `validateStatus` پاسخ با status
`TIMEOUT`	زمان درخواست تمام شده است
`NETWORK_ERROR`	خطای ارتباط با transport
`ABORTED`	درخواست توسط کاربر یا `abortKey` لغو شده است
`PARSE_ERROR`	بدنه پاسخ JSON معتبر نیست
`INVALID_REQUEST`	تنظیمات، transport، URL یا بدنه درخواست نامعتبر است
`UNTRUSTED_ORIGIN`	ارسال اعتبارنامه به origin غیرمجاز درخواست شده است

برای تغییر معیار پذیرش status از `validateStatus(status)` استفاده کنید. مقدار پیش‌فرض statusهای 200 تا 299 را می‌پذیرد.

```
try {
  await api.get("/users/404");
} catch (error) {
  if (error instanceof ApiError) {
    console.error(error.code, error.status, error.data);
  }
}
```

Interceptorها و ثبت رخداد

هر stack متدهای `clear` ، `eject(id)` ، `use(fn)()` و `list()` دارد:

```
api.interceptors.request.use((config) => ({
  ...config,
  headers: { ...config.headers, "X-App-Version": "2" },
}));

api.interceptors.response.use((response) => {
  response.data.receivedAt = Date.now();
  return response;
});
```

```
api.interceptors.error.use((error) => {
  if (error.status === 404) return null; // بازیابی خطا با مقدار دلخواه
});
```

پاسخ شیء پاسخ کامل را می‌گیرد و می‌تواند نسخه Interceptor درخواست را تغییر می‌دهد، config را می‌گیرد؛ با برگرداندن config درخواست Interceptor برگرداند، پردازش `undefined` جدید برگرداند یا خطا را با مقدار دیگری بازیابی کند. اگر `Error` خطا می‌تواند یک Interceptor. تغییریافته آن را برگرداند. خطا ادامه می‌یابد.

رخدادها به‌شکل JSON ساخت‌یافته ثبت می‌شوند. سطح پیش‌فرض `warn` است. کلیدهایی مانند api key، authorization، secret، token، password، cookie و credential در metadata پاک‌سازی می‌شوند؛ پارامترهای حساس URL نیز پوشانده می‌شوند. این پاک‌سازی جایگزین حذف اطلاعات حساس از داده‌های خود برنامه نیست.

Transport سفارشی

تابع transport باید یک Promise از شیء پاسخ خام برگرداند. statusهای HTTP باید resolve شوند؛ فقط خطای شبکه یا لغو باید reject شود.

```
const api = createClient({
  baseURL: "https://api.example.com",
  transport: async (request) => {
    // request شامل url، method، headers، body، credentials، signal و responseType است.
    return {
      status: 200,
      statusText: "OK",
      headers: { "content-type": "application/json" },
      body: JSON.stringify({ ok: true }),
    };
  },
});
```

استفاده کنید؛ در حالت XHR بارگذاری یا دریافت، از progress پشتیبانی نمی‌شود. برای `arraybuffer` و `blob` برای پاسخ‌های Transport jQuery وجود `auto`، callbackهای progress باعث انتخاب XHR می‌شود.

پلاگین‌ها

پلاگین‌ها جداگانه از `base-ajax/plugins` وارد می‌شوند. هر پلاگین یک client جدید می‌سازد و client اصلی را تغییر نمی‌دهد.

Cache

`cache(client, options)` پاسخ متدهای مشخص را در حافظه نگه می‌دارد. پیش‌فرض فقط GET را cache می‌کند و TTL برابر ۶۰ ثانیه است. `staleWhileRevalidate: true` داده منقضی‌شده را فوراً برمی‌گرداند و به‌روزرسانی را در پس‌زمینه انجام می‌دهد.

```
const cachedApi = cache(api, { ttl: 30_000, staleWhileRevalidate: true });
await cachedApi.get("/users");
cachedApi.cache.clear();
```

گزینه‌های دیگر: `keyFn(config)`، `methods` و `store` از نوع handle. `Map` به نام `cache` متدهای `get`، `has`، `delete` و `clear` دارد. پاسخ‌ها با reference مشترک نگهداری می‌شوند؛ اگر مصرف‌کننده آن‌ها را تغییر دهد، مقدار cache نیز تغییر می‌کند. `keys`، `size` و `key` به‌تنهایی missهای همزمان را ادغام نمی‌کند؛ برای آن `dedupe` را هم اضافه کنید.

Dedupe

`dedupe(client, options)` درخواست‌های همزمان با کلید یکسان را در یک Promise مشترک ادغام می‌کند. پیش‌فرض متدهای GET و HEAD را بررسی می‌کند. درخواست‌های بعدی پس از پایان درخواست قبلی دوباره به شبکه می‌روند.

```
const dedupedApi = dedupe(api);
const [a, b] = await Promise.all([
  dedupedApi.get("/users"),
  dedupedApi.get("/users"),
]);
```

گزینه‌ها: `methods` و `keyFn(config)`. مقدار `inflightSize` تعداد کلیدهای در حال اجرا را نشان می‌دهد.

Queue

`queue(client, options)` تعداد درخواست‌های همزمان را محدود می‌کند. پیش‌فرض `maxConcurrent` برابر ۶ است. درخواست‌های دارای `priority` بالاتر زودتر از صف اجرا می‌شوند؛ پیش‌فرض اولویت صفر است.

```
const queuedApi = queue(api, { maxConcurrent: 4 });
queuedApi.get("/urgent", undefined, { priority: 10 });
console.log(queuedApi.queue.active, queuedApi.queue.pending);
```

`priority` فقط hint پلاگین است و پیش از رسیدن درخواست به هسته حذف می‌شود. handle به نام `queue` مقادیر `active`، `pending` و `maxConcurrent` را ارائه می‌دهد.

Circuit breaker

`circuitBreaker(client, options)` حالت‌ها پس از رسیدن تعداد خطاهای متوالی به آستانه، درخواست‌ها را موقتاً رد می‌کند. حالت‌ها `closed`، `open` و `half-open` هستند. پیش‌فرض‌ها: `failureThreshold: 5`، `successThreshold: 2` و `cooldownMs: 30000`.

```
const resilientApi = circuitBreaker(api, {
  failureThreshold: 5,
  cooldownMs: 15_000,
});
console.log(resilientApi.circuitBreaker.getState());
```

گزینه `shouldCount(error)` تعیین می‌کند کدام خطاها شمرده شوند؛ حالت پیش‌فرض خطاهای لغوشده را نادیده می‌گیرد. callbackهای `onOpen`، `onHalfOpen` و `onClose` برای آگاه‌شدن از تغییر حالت هستند. handle متدهای `getStats`، `getState` و `reset` دارد. هنگام بازبودن مدار، خطای `CircuitOpenError` رد می‌شود.

Trace

`trace(client, options)` header مربوط به W3C Trace Context نام پیش‌فرض `traceparent` برابر header را به درخواست اضافه می‌کند. اگر caller از قبل آن header را داده باشد، پلاگین آن را بازنویسی نمی‌کند.

```
const tracedApi = trace(api, {
  onTrace: (value) => console.debug(value),
});
```

گزینه‌ها: `getTraceparent`، `headerName` (`()`)، `validate` برای تأمین مقدار upstream، `onTrace(value)` و `onInvalid(value)`. تابع `generateTraceparent()` نیز مقدار جدید با قالب W3C تولید می‌کند. اعتبارسنجی پیش‌فرض فقط callback اختیاری `onInvalid` را فراخوانی می‌کند و درخواست را متوقف نمی‌کند.

Mock

`mock(handlers)` یک transport آزمایشی می‌سازد. هر handler شامل `match` و `response` است. `match` می‌تواند URL دقیق، رشته `"METHOD URL"`، RegExp یا تابع `(request) => boolean` باشد. `response` می‌تواند شیء پاسخ یا تابع همگام/ناهمگام باشد.

```
const testApi = createClient({
  origin: "https://test.example",
  transport: mock([
    {
      match: "GET https://test.example/users",
      response: { body: [{ id: 1 }] },
    },
    {
      match: /\/users\/\d+$/,
      response: (request) => ({ body: { url: request.url } }),
    },
  ]),
});
```

اگر status مشخص نشود، مقدار 200 استفاده می‌شود. header پیش‌فرض `Content-Type` برابر `application/json` است. route پیدانشده پاسخ خام 404 می‌دهد که با `validateStatus` پیش‌فرض به خطای `HTTP_ERROR` تبدیل می‌شود.

ترکیب پلاگین‌ها

`compose(client, ...plugins)` پلاگین‌ها را از چپ به راست اعمال می‌کند. پلاگین اول بیرونی‌ترین لایه است؛ بنابراین ترتیب می‌تواند رفتار و محدوده اثر هر پلاگین را تغییر دهد.

```
const enhancedApi = compose(
  api,
  (client) => queue(client, { maxConcurrent: 4 }),
  (client) => dedupe(client),
  (client) => cache(client, { ttl: 30_000 }),
  (client) => circuitBreaker(client, { failureThreshold: 5 }),
  (client) => trace(client),
);
```

ساخت و بررسی پروژه

برای ساخت خروجی‌ها:

```
npm run build
```

جداگانه UMD و CommonJS، ESM می‌سازد. پلاگین‌ها نیز خروجی `dist/` فشرده در UMD و UMD، CommonJS، ESM هسته را به Rollup دارند. Source mapها هم تولید می‌شوند.

فرمان‌های توسعه:

فرمان	کاربرد
`npm test`	اجرای همه تست‌های Vitest
`npm run test:watch`	اجرای Vitest در حالت watch
`npm run test:coverage`	اجرای تست‌ها همراه پوشش کد
`npm run test:property`	اجرای تست‌های property در `tests/property`
`npm run test:types`	بررسی تست‌های نوع TypeScript
`npm run typecheck`	بررسی نوع فایل‌های پروژه با TypeScript
`npm run verify`	اجرای typecheck، تست‌های پوشش‌دار و build
`npm run bench`	ساخت خروجی و اجرای benchmark هسته

فرمان	کاربرد
`npm run serve:examples`	اجرای سرور محلی مثال‌ها روی پورت 5173

برای جزئیات API و نوع‌ها به تعریف‌های BaseAjax و تعریف‌های پلاگین‌ها مراجعه کنید.